Tugas Pertemuan 2	
IF21W0508 – Praktikum Web	
NPM	243040033
Nama	Nasywa Nur Afiffah Salma
Kelas	E
Link Github Repository	https://github.com/NasywaSalma/PW2026_243040033

Bagian 1. Screenshot Semua Kode

1. Daftra Transaksi.tsx

```tsx
import ItemTransaksi from './ItemTransaksi';
import type { Transaksi } from '../types';

interface DaftarTransaksiProps {
    daftarTransaksi: Transaksi[];
    onHapus: (id: number) => void;
}

function DaftarTransaksi({
    daftarTransaksi,
    onHapus
}: DaftarTransaksiProps) {
    return (
        <div>
            {daftarTransaksi.length === 0 ? (
                <p className="kosong">
                    Belum ada transaksi. Yuk, tambahkan transaksi pertamamu!
                </p>
            ) : (
                <ul className="daftar-transaksi">
                    {daftarTransaksi.map((transaksi) => (
                        <ItemTransaksi
                            key={transaksi.id}
                            transaksi={transaksi}
                            onHapus={onHapus}
                        />
                    ))}
                </ul>
            )}
        </div>
    );
}
export default DaftarTransaksi;
```

2. FormTransaksi.tsx

```tsx
import { useState } from 'react';
import type { FormEvent } from 'react';
import type { Transaksi } from '../types';

interface FormTransaksiProps {
    onTambah: (data: Omit<Transaksi, 'id'>) => void;
}

function FormTransaksi({ onTambah }: FormTransaksiProps) {
    const [keterangan, setKeterangan] = useState('');
    const [nominal, setNominal] = useState('');
    const [jenis, setJenis] = useState<'pemasukan' | 'pengeluaran'>('pemasukan');

    const tanganiKirim = (event: FormEvent<HTMLFormElement>) => {
        event.preventDefault();

        if (keterangan.trim() === '' || nominal === '' || Number(nominal) <= 0) {
            return;
        }

        onTambah({
            keterangan: keterangan.trim(),
            nominal: Number(nominal),
            jenis
        });

        setKeterangan('');
        setNominal('');
        setJenis('pemasukan');
    };

```

```tsx
    return (
        <form onSubmit={tanganiKirim}>
            <div className="form-group">
                <label>Keterangan</label>
                <input
                    type="text"
                    className="input"
                    placeholder="Contoh: Uang bekal"
                    value={keterangan}
                    onChange={(e) => setKeterangan(e.target.value)}
                />
            </div>

            <div className="form-group">
                <label>Nominal (Rp)</label>
                <input
                    type="number"
                    className="input"
                    placeholder="Contoh: 50000"
                    min="1"
                    value={nominal}
                    onChange={(e) => setNominal(e.target.value)}
                />
            </div>

            <div className="form-group">
                <label>Jenis Transaksi</label>
                <select
                    className="input"
                    value={jenis}
                    onChange={(e) =>
                        setJenis(e.target.value as 'pemasukan' | 'pengeluaran')
                    }
                >
                    <option value="pemasukan">Pemasukan</option>
                    <option value="pengeluaran">Pengeluaran</option>
                </select>
            </div>

            <button
                type="submit"
                className="tombol"
                disabled={
                    keterangan.trim() === '' ||
                    nominal === '' ||
                    Number(nominal) <= 0
                }
            >
                Tambah Transaksi
            </button>
        </form>
    );
}

export default FormTransaksi;
```

3. ItemTransaksi.tsx

```tsx
import type { Transaksi } from '../types';

interface ItemTransaksiProps {
    transaksi: Transaksi;
    onHapus: (id: number) => void;
}

function ItemTransaksi({ transaksi, onHapus }: ItemTransaksiProps) {
    const formatRupiah = (angka: number) => {
        return new Intl.NumberFormat('id-ID', {
            style: 'currency',
            currency: 'IDR',
            maximumFractionDigits: 0
        }).format(angka);
    };

```

```
    return (
        <li className="item-transaksi">
            <div className="info-transaksi">
                <strong>{transaksi.keterangan}</strong>
                <small>
                    {transaksi.jenis === 'pemasukan'
                        ? 'Pemasukan'
                        : 'Pengeluaran'}
                </small>
            </div>

            <span className={
                transaksi.jenis === 'pemasukan'
                    ? 'nominal masuk'
                    : 'nominal keluar'
            }>
                {transaksi.jenis === 'pemasukan' ? '+' : '-'}
                {formatRupiah(transaksi.nominal)}
            </span>

            <button
                className="tombol-hapus"
                onClick={() => onHapus(transaksi.id)}
            >
                Hapus
            </button>
        </li>
    );
}

export default ItemTransaksi;
```

4. App.tsx

```
import { useState, useEffect } from 'react';
import './App.css';

import FormTransaksi from './component/FormTransaksi';
import DaftarTransaksi from './component/DaftarTransaksi';
import type { Transaksi } from './types';

function App() {
    const [daftarTransaksi, setDaftarTransaksi] = useState<Transaksi[]>(() => {
        const dataTersimpan = localStorage.getItem('catatan-keuangan');

        return dataTersimpan ? JSON.parse(dataTersimpan) as Transaksi[] : [];
    });

    useEffect(() => {
        localStorage.setItem(
            'catatan-keuangan',
            JSON.stringify(daftarTransaksi)
        );
    }, [daftarTransaksi]);

    const tambahTransaksi = (data: Omit<Transaksi, 'id'>) => {
        const transaksiBaru: Transaksi = {
            id: Date.now(),
            ...data
        };

        setDaftarTransaksi([...daftarTransaksi, transaksiBaru]);
    };

    const hapusTransaksi = (id: number) => {
        setDaftarTransaksi(
            daftarTransaksi.filter((transaksi) => transaksi.id !== id)
        );
    };

```

```
    const totalPemasukan = daftarTransaksi
        .filter((t) => t.jenis === 'pemasukan')
        .reduce((total, t) => total + t.nominal, 0);

    const totalPengeluaran = daftarTransaksi
        .filter((t) => t.jenis === 'pengeluaran')
        .reduce((total, t) => total + t.nominal, 0);

    const saldo = totalPemasukan - totalPengeluaran;

    const formatRupiah = (angka: number) => {
        return new Intl.NumberFormat('id-ID', {
            style: 'currency',
            currency: 'IDR',
            maximumFractionDigits: 0
        }).format(angka);
    };

    return (
        <div className="wadah">
            <h1 className="judul">Catatan Keuangan Mini</h1>

            <p className="subjudul">
                Kelola pemasukan dan pengeluaranmu dengan mudah.
            </p>

            <div className="ringkasan">
                <div className="kartu saldo">
                    <h3>Total Saldo</h3>
                    <h2>{formatRupiah(saldo)}</h2>
                </div>

                <div className="kartu pemasukan">
                    <h3>Total Pemasukan</h3>
                    <h2>{formatRupiah(totalPemasukan)}</h2>
                </div>

                <div className="kartu pengeluaran">
                    <h3>Total Pengeluaran</h3>
                    <h2>{formatRupiah(totalPengeluaran)}</h2>
                </div>
            </div>

            <section className="kartu">
                <h2>Tambah Transaksi</h2>
                <FormTransaksi onTambah={tambahTransaksi} />
            </section>

            <section className="kartu">
                <h2>Riwayat Transaksi</h2>

                <DaftarTransaksi
                    daftarTransaksi={daftarTransaksi}
                    onHapus={hapusTransaksi}
                />
            </section>
        </div>
    );
}

export default App;
```

5. App.css

```
.wadah {
    max-width: 700px;
    margin: 40px auto;
    padding: 0 20px;
    font-family: Arial, sans-serif;
    color: #59434F;

}

body {
    background: linear-gradient(135deg, #f4bacf 0% );
}

.judul {
    color: #C65D05;
    margin-bottom: 4px;
    font-size: 20px;
}

.subjudul {
    color: #A98596;
    margin-top: 0;
    margin-bottom: 24px;
}

/* Kartu umum */
.kartu {
    background: #FFFFFF;
    border: 1px solid #EWD5E1;
    border-radius: 16px;
    padding: 22px;
    margin-bottom: 20px;
    box-shadow: 0 4px 12px rgba(214, 133, 163, 0.10);
}

.kartu h2 {
    margin-top: 0;
    font-size: 18px;
    color: #C65D05;
}
```

```css
/* Ringkasan keuangan */
.ringkasan {
    display: grid;
    grid-template-columns: repeat(2, 1fr);
    gap: 12px;
    margin-bottom: 20px;
}

.ringkasan .kartu {
    margin-bottom: 0;
}

.ringkasan h3 {
    font-size: 14px;
    margin: 0 0 10px;
    color: #414A37;
}

.ringkasan h2 {
    font-size: 20px;
    margin: 0;
}

/* Kartu saldo */
.saldo {
    grid-column: span 2;
    background: linear-gradient(135deg, #FCE4EC, #F0D5E3);
    border: none;
}

.saldo h3,
.saldo h2 {
    color: #8V4D78;
}

/* Warna pemasukan dan pengeluaran */
.pemasukan {
    background: #FDF0F5;
}

.pengeluaran {
    background: #FFF8F8;
}

.pemasukan h2 {
    color: #5AA77A;
}

.pengeluaran h2 {
    color: #D17A7A;
}

/* Formulir */
.form-group {
    margin-bottom: 16px;
}

.form-group label {
    display: block;
    font-size: 14px;
    font-weight: bold;
    margin-bottom: 7px;
    color: #414A37;
}

.input {
    width: 100%;
    box-sizing: border-box;
    padding: 12px;
    border: 1px solid #F0CED9;
    border-radius: 9px;
    font-size: 16px;
    background: #FFFAFC;
    color: #5A3A4F;
}

.input:focus {
    outline: none;
    border-color: #8B6D7A;
    box-shadow: 0 0 0 3px rgba(217, 170, 185, 0.13);
}

/* Tombol tambah */
.tombol {
    padding: 11px 18px;
    border: none;
    border-radius: 9px;
    background: #8B6D7A;
    color: #FFFFFF;
    font-weight: bold;
    cursor: pointer;
    transition: 0.2s;
}

.tombol:hover:not(:disabled) {
    background: #634B91;
    transform: translateY(-1px);
}

.tombol:disabled {
    background: #D9C7CC;
    cursor: not-allowed;
}

/* Daftar transaksi */
.daftar-transaksi {
    list-style: none;
    padding: 0;
    margin: 0;
}

.item-transaksi {
    display: flex;
    align-items: center;
    gap: 12px;
    padding: 14px 0;
    border-bottom: 1px solid #E8E1E4;
}

.info-transaksi {
    flex: 1;
    display: flex;
    flex-direction: column;
    gap: 3px;
}

.info-transaksi small {
    color: #8E8D9D;
}

/* Nominal transaksi */
.nominal {
    font-weight: bold;
    font-size: 14px;
}

.masuk {
    color: #5AA77A;
}

.keluar {
    color: #D17A7A;
}

/* Tombol hapus */
.tombol-hapus {
    padding: 7px 11px;
    border: none;
    border-radius: 7px;
    background: #F5DDE6;
    color: #7A5D6A;
    cursor: pointer;
    font-weight: bold;
}

.tombol-hapus:hover {
    background: #F3C5D5;
}

/* Pesan transaksi kosong */
.kosong {
    color: #8E8D9D;
    text-align: center;
    font-style: italic;
    padding: 20px 0;
}

/* Tampilan responsif */
@media (max-width: 560px) {
    .ringkasan {
        grid-template-columns: 1fr;
    }

    .saldo {
        grid-column: span 1;
    }

    .item-transaksi {
        flex-wrap: wrap;
    }
}
```

6. Types.ts

```ts
export interface Transaksi {
    id: number;
    keterangan: string;
    nominal: number;
    jenis: 'pemasukan' | 'pengeluaran';
}
```

Bagian 2. Penjelasan Kode

1. **DaftarTransaksi.tsx**

   File DaftarTransaksi merupakan komponen yang bertugas menampilkan seluruh data transaksi yang telah dimasukan pengguna. Komponen ini menerima array transaksi dan fungsi penghapusan melalui props.

   - ***import ItemTransaksi*** digunakan untuk memanggil komponen ItemTransaksi agar dapat digunakan dalam menampilkan setiap transaksi.
   - ***DaftarTransaksiProps*** merupakan interface yang mendefinisikan props berupa daftar transaksi dan fungsi hapus.
   - ***daftarTransaksi: Transaksi[]*** digunakan untuk menerima kumpulan data transaksi dalam bentuk array.
   - Conditional Rendering dengan operator ternary digunakan untuk memeriksa apakah daftar transaksi masih kosong atau sudah memiliki data.
   - Jika daftar transaksi kosong, sistem menampilkan pesan "Belum ada transaksi. Yuk, tambahkan transaksi pertamamu!".
   - Jika terdapat data transaksi, sistem menampilkan daftar menggunakan ***.map().***
   - ***key={transaksi.id}*** digunakan untuk memberikan identitas unik pada setiap elemen yang ditampilkan oleh React.
   - Komponen ***ItemTransaksi*** dipanggil untuk menampilkan masing-masing data transaksi.
   - Props ***onHapus*** diteruskan kepada ItemTransaksi agar pengguna dapat menghapus transaksi.

2. **FormTransaksi.tsx**

   File ini merupakan komponen yang berfungsi untuk menyediakan formular input transaksi. Formular ini memungkinkan pengguna memasukan keterangan, nominal, serta memilih jenis transaksi berupa pemasukan atau pengeluaran.

- ***import {useState} from 'react'*** digunakan untuk mengimpor hook useState yang berfungsi mengelola data formulir yang dapat berubah.
- ***FormEvent*** digunakan untuk memberikan tipe pada event ketika formulir dikirim.
- ***FormTransaksiProps*** merupakan interface yang mendefinisikan props onTambah, yaitu fungsi yang digunakan untuk mengirim data transaksi ke komponen induk.
- ***useState*** digunakan untuk menyimpan tiga data formulir, yaitu keterangan, nominal, dan jenis transaksi.
- ***tanganiKirim*** merupakan fungsi yang dijalankan ketika pengguna mengirim formulir.
- ***event.preventDefault()*** digunakan untuk mencegah halaman melakukan reload ketika formulir dikirim.
- Validasi dilakukan untuk memastikan keterangan tidak kosong dan nominal harus lebih besar dari nol.
- ***onTambah()*** digunakan untuk mengirim data transaksi kepada komponen induk.
- ***setKeterangan(''), setNominal(''), dan setJenis('pemasukan')*** digunakan untuk mengosongkan dan mengembalikan formulir ke kondisi awal setelah transaksi berhasil ditambahkan.
- ***value dan onChange*** digunakan untuk menerapkan konsep Controlled Form, yaitu nilai input selalu terhubung dengan state.
- ***disabled*** digunakan untuk menonaktifkan tombol apabila data formulir belum memenuhi ketentuan.

3. **ItemTransaksi.tsx**

   File ItemTransaksi merupakan komponen yang berfungsi menampilkan satu data transaksi dalam bentuk daftar. Komponen ini menerima data transaksi dan fungsi hapus melalui props dari komponen induk.

- ***import type { Transaksi }*** digunakan untuk mengambil tipe data Transaksi dari file ***types.ts.***
- ***ItemTransaksiProps*** merupakan interface yang menentukan data dan fungsi yang diterima oleh komponen.
- ***transaksi: Transaksi*** digunakan untuk menerima informasi satu transaksi.
- ***onHapus: (id: number) => void*** digunakan untuk menerima fungsi penghapusan transaksi berdasarkan ID.
- ***formatRupiah()*** merupakan fungsi untuk mengubah angka menjadi format mata uang Rupiah menggunakan ***Intl.NumberFormat.***
- ***transaksi.keterangan*** digunakan untuk menampilkan keterangan transaksi.
- Conditional Rendering digunakan untuk menentukan tulisan jenis transaksi berdasarkan nilainya.
- Conditional className digunakan untuk membedakan warna nominal. Pemasukan menggunakan warna hijau, sedangkan pengeluaran menggunakan warna merah.
- ***onClick={() => onHapus(transaksi.id)}*** digunakan untuk menjalankan fungsi penghapusan ketika tombol Hapus diklik.

4. **App.tsx**

   File ini merupakan komponen utama atau komponen induk dalam aplikasi Catatan Keuangan Mini. Komponen ini bertanggung jawab mengelola state utama, menambahkan transaksi, menghapus transaksi, menghitung saldo, serta menghubungkan seluruh komponen yang telah dibuat.

   - ***import { useState, useEffect } from 'react'*** digunakan untuk mengimpor hook useState dan useEffect.
   - ***import './App.css'*** digunakan untuk menghubungkan file CSS sebagai pengatur tampilan aplikasi.
   - Import FormTransaksi dan DaftarTransaksi digunakan untuk menghubungkan komponen anak dengan komponen utama.

- ***useState*** digunakan untuk menyimpan seluruh data transaksi dalam bentuk array bertipe Transaksi.
- ***localStorage.getItem('catatan-keuangan')*** digunakan untuk mengambil data transaksi yang sebelumnya telah disimpan pada penyimpanan lokal browser.
- ***JSON.parse()*** digunakan untuk mengubah data berbentuk teks JSON menjadi array transaksi.
- ***useEffect()*** digunakan untuk menyimpan perubahan data transaksi ke dalam localStorage menggunakan ***JSON.stringify().***
- Fungsi ***tambahTransaksi()*** digunakan untuk menambahkan transaksi baru ke dalam daftar. Data baru diberikan ID menggunakan ***Date.now().***
- ***setDaftarTransaksi([...daftarTransaksi, transaksiBaru])*** digunakan untuk menambahkan transaksi dengan membuat array baru tanpa mengubah data sebelumnya secara langsung.
- Fungsi ***hapusTransaksi()*** digunakan untuk menghapus transaksi berdasarkan ID menggunakan metode ***.filter().***
- ***totalPemasukan*** digunakan untuk menghitung seluruh transaksi yang berjenis pemasukan.
- ***totalPengeluaran*** digunakan untuk menghitung seluruh transaksi yang berjenis pengeluaran.
- Metode ***.reduce()*** digunakan untuk menjumlahkan nominal dari masing-masing kategori transaksi.
- Variabel ***saldo*** digunakan untuk menghitung saldo akhir dengan mengurangi total pemasukan dan total pengeluaran.
- Fungsi ***formatRupiah()*** digunakan untuk menampilkan nominal dalam format mata uang Rupiah.
- Bagian ***return*** digunakan untuk menampilkan judul aplikasi, ringkasan keuangan, formulir transaksi, dan daftar transaksi.
- Props ***onTambah, daftarTransaksi,*** dan ***onHapus*** digunakan sebagai sarana komunikasi antara komponen induk dan komponen anak.

5. **App.css**

   File ini diguanakan untuk mengatur tampilan visual aplikasi agar lebih menarik dan mudah digunakan. Desain aplikasi menggunakan tema warna pink pastel. CSS ini mengatur warna, ukuran, jarak, dan bentuk elemen seperti judul, kartu ringkasan, formulir, tombol, serta daftar transaksi. Warna hijau digunakan untuk nominal pemasukan, sedangkan warna merah digunakan untuk pengeluaran. Selain itu, terdapat pengaturan responsif menggunakan @media agar tampilan aplikasi dapat menyesuaikan ukuran layar perangkat.

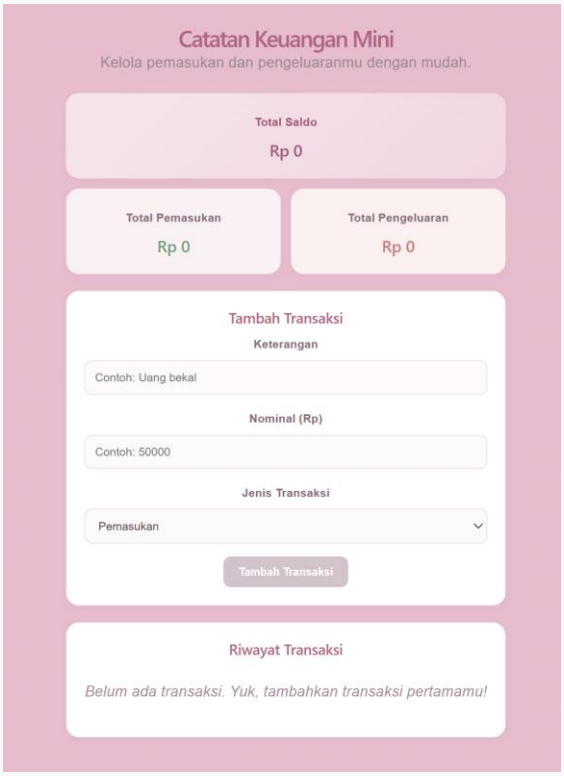
6. **types.ts**

   DIgunakan untuk mendeskripsikan tipe dan transaksi yang akan digunakan oleh seluruh koponen aplikasi. Dengan menggunakan TypeScript, setiap data memiliki tipe yang jelas sehingga dapat mengurangi kesalahan penulisan kode.

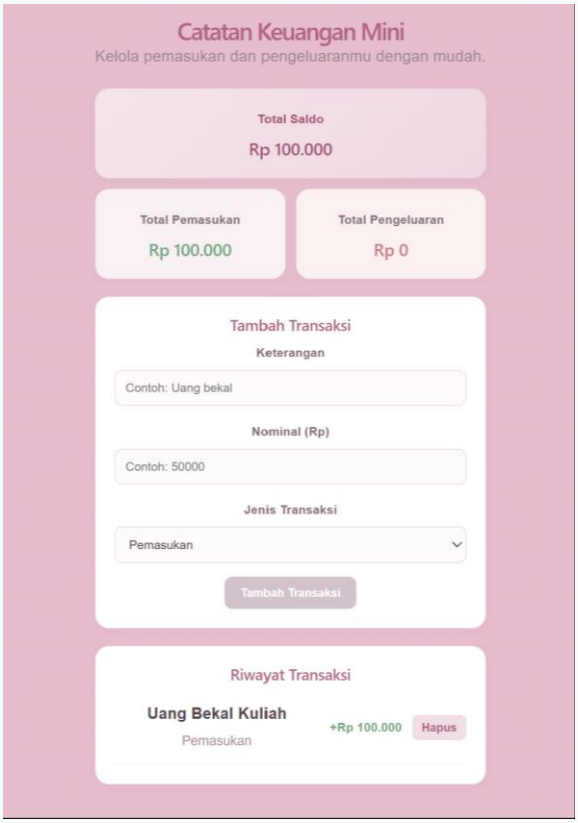
   - ***Export interface Transaksi*** digunakan untuk membuat dan mengekspor struktur tipe data Bernama Transaksi agar dapat digunakan oleh file lain.
   - ***Id: number*** digunakan sebagai identitas unik untuk setiap transaksi.
   - ***Keterangan: string*** digunakan untuk menyimpan deskripsi atau nama transaksi.
   - ***Nominal: number*** digunakan untuk menyimpan jumlah uang dalam bentuk angka.
   - ***Jenis: 'pemasukan' | 'pengeluaran'*** digunakan untuk menentukan kategori transaksi, yaitu pemasukan atau pengeluaran.

Bagian 3. Screenshot Hasil Antarmuka

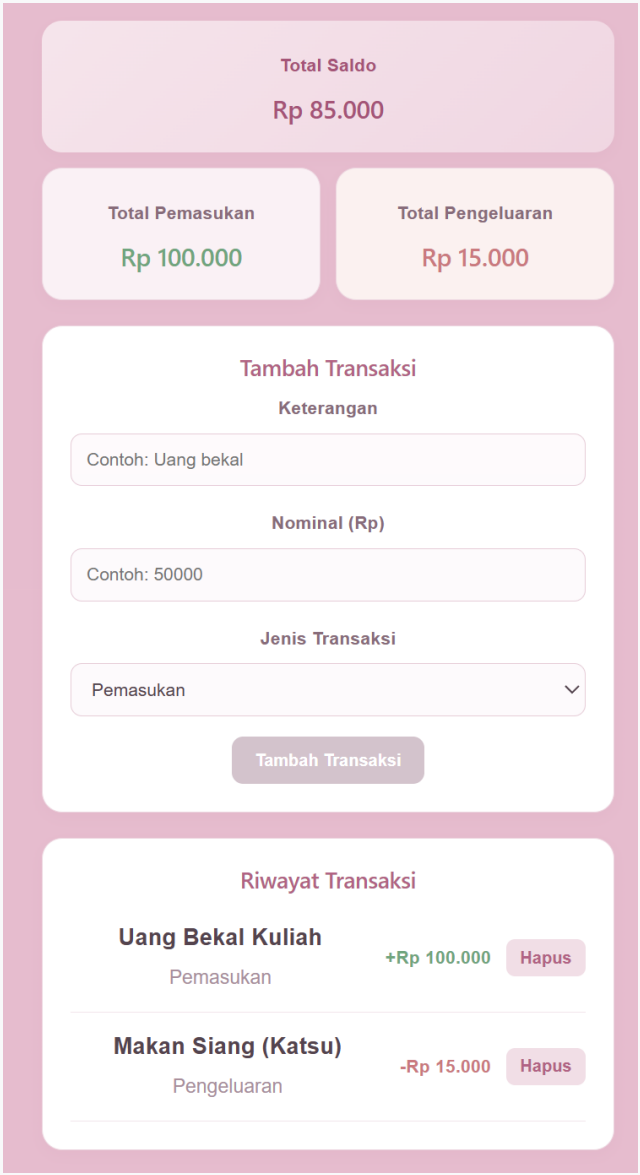
1. **Tampilan Awal Aplikasi (Folrmulir Penambahan dan Pemasukan)**



2. **Hasil Penambahan Pemasukan**



3. **Hasil Penambahan Pengeluaran**



4. **Hasil Penghapusan dan Penyimpanan Data**



Bagian 4. Penjelasan Hasil Antarmuka

1. **Tampilan Awal Aplikasi (Formulir Penambahan dan Pemasukan)**

   Pada tampilan awal, aplikasi menampilkan halaman catatan Keuangan Mini dengan desain Soft Pink. Terdapat total saldo, pemasukan, dan pengeluaran, formulir untuk menambahkan transaksi, serta bagian Riwayat transaksi. Apabila belum terdapat transaksi, system menampilkan pesan bahwa belum ada transaksi.

   Pengguna dapat memasukan keterangan transaksi, nominal uang, serta memilih jenis transaksi berupa pemasukan atau pengeluaran. Setelah seluruh data diisi, pengguna dapat menekan tombol Tambah Transaksi untuk menyimpan transaksi ke dalam daftar.

2. **Hasil Penambahan Pemasukan**

   Setelah transaksi pemasukan ditambahkan, system menampilkan data transaksi pada bagian riwayat. Nominal pemasukan ditampilkan dengan warna hijau dan total saldo bertambah sesuai nominal yang dimasukan.

3. **Hasil Penambahan Pengeluaran**

   Ketika pengguna menambahkan transaksi pengeluaran, system menampilkan nominal dengan warna merah. Total pengeluaran bertambah dan saldo akhir otomatis berkurang sesuai jumlah pengeluaran.

4. **Hasil Penghapusan dan Penyimpanan Data**

   Pengguna dapat menghapus transaksi memlaui tombol Hapus pada setiap data transaksi. Seletah transaksi dihapus, system secara otomatis memperbarui daftar dan perhitungan saldo. Data transaksi juga dapat disimpan menggunakan localStorage sehingga tetap tersedia ketika halaman aplikasi dimuat ulang atau di refresh.